

AdaptiveISP 实验报告

孙瑞泽

复现与分析：基于强化学习的自适应 ISP 管线

摘要：本文基于 AdaptiveISP 开源代码完成环境部署、自建鸟类 RAW 数据集构建与基线复现，完整解析强化学习 MDP 建模、ISP 滤波器、训练更新流程。设计两组对比实验：1) 使用 LOD 预训练 Agent 在鸟类数据集微调，验证单场景小数据集易出现策略过拟合；2) 消融 ISP 最大迭代步数（5 步缩减至 3 步），量化精度与推理速度的权衡关系。总结实验中显存溢出、伪标签缺陷、奖励震荡等问题，并提出数据增强、奖励函数优化等改进方案。

实验环境：RTX 4060 Laptop GPU (8GB VRAM), PyTorch 2.0.1, CUDA 11.8

数据集：自建 BIRDS 鸟类数据集 (500 张 ARW $\rightarrow$ PNG, YOLOv8 pseudo-label)

一、环境配置与基线复现

1.1 环境配置

硬件：联想 Legion R9000P，NVIDIA GeForce RTX 4060 Laptop GPU (8GB VRAM)，AMD Ryzen 7 7745HX，16GB RAM。

软件：Ubuntu 22.04, Miniconda, Python 3.10.20, PyTorch 2.0.1+cu118。

安装步骤：按照仓库根目录 requirements.txt 创建 conda 环境 "adaptiveisp"，执行 setup.sh 完成依赖安装。

1.2 数据集准备

原始数据：500 张 Sony ARW 格式的鸟类照片，使用 rawpy + gamma 校正转换为 PNG。

标注：使用 YOLOv8x 预训练模型（COCO 80 类）对 PNG 图片进行推理，以置信度 ≥ 0.5 的检测结果作为 pseudo-label，导出为 YOLO 格式的标签文件。

数据集划分：400 张训练集，100 张验证集，存放于 datasets/BIRDS/PNG/，配置为 yolov3/data/bird.yaml。

注：pseudo-label 共 80 个 COCO 类别，但 BIRDS 数据集中主要出现的类别为 bird (14)、person (0)、bicycle (1) 等。

1.3 基线测试

使用论文提供的预训练 LOD-Agent (ckpt-lod-df-1.0.pth) + YOLOv3 检测器，在 BIRDS val 100 上评测，作为后续实验的对比基线。

命令：

```
python yolov3/val_adaptiveisp.py --data=yolov3/data/bird.yaml --data_name=lod --isp_model=Agent --isp_weights=./pretrained/ckpt-lod-df-1.0.pth --weights=./pretrained/yolov3.pt --cfg_file=config --steps=5 --batch-size=1
```

基线结果：

模型	mAP50	mAP75	mAP50-95	Precision	Recall	推理时间
LOD-Agent (steps=5)	0.738	0.579	0.551	0.735	0.698	374.4ms

1.4 遇到的问题与解决

问题 1：验证阶段 OOM。训练时显存稳定在 3.48GB，但验证代码一次性加载 100 张图并行跑 10 个滤波器，当某张图触发 NLM (Non-Local Means Denoise) 时，额外分配 2-3GB 张量导致超出 8GB 上限。

解决：将 `config.py` 中 `val_freq` 设置为 999999，跳过验证阶段，仅保留训练和 `checkpoint` 保存。训练结束后单独运行评测脚本。

问题 2：BIRDS 数据集格式兼容。`val_adaptiveisp.py` 中 `data_name` 参数仅支持 "coco" 和 "lod/oprd/rod"。BIRDS 采用了与 LOD 相同的目录结构（images + labels + txt 列表），因此将 `data_name` 设为 lod 即可正常加载。

问题 3：YOLOv3 预训练权重类数不匹配。`bird.yaml` 中 `nc=80`，与 `yolov3.pt` 一致，无需修改。但若未来使用自定义类别数，需要相应调整 `yaml` 和模型。

二、关键模块理解

2.1 强化学习建模

AdaptiveISP 将 ISP 管线优化建模为马尔可夫决策过程 (MDP):

RL 概念	代码对应	说明
State	当前图像 (64×64) + 步数 + 滤波器使用历史 + 停止信号	13 维状态向量 (3 + 10 filters)
Action	从 10 个 ISP 滤波器中选择 1 个	离散动作空间, 维度 = len(cfg.filters) = 10
Reward	(检测损失降低量) × 100 - penalty	critic_logit_multiplier = 100
Policy (Agent)	agent.py: Agent 类	CNN 编码图像 → FC → 10 维 action 概率分布 → 采样
Value	value.py: Value 类	CNN 编码图像+状态 → 标量 V(s), 用于 TD 学习

2.2 ISP 滤波器

共 10 个 ISP 操作, 每个滤波器含参数预测器 (filter_param_regressor) 和图像处理 (process) 两个组件:

缩写	模块	参数数	耗时 (ms)	功能
E	ExposureFilter	1	1.7	全局曝光调整
G	GammaFilter	1	2.0	Gamma 校正, 参数 ∈ [1/3, 3]
CCM	CCMFilter	9	1.9	3×3 色彩校正矩阵
Shr	SharpenFilter	1	6.3	USM 锐化
NLM	DenoiseFilter	1	10.0	非局部均值降噪 (最耗时)
T	ToneFilter	8	2.7	8 点色调曲线调整
Ct	ContrastFilter	1	2.1	对比度调整
S+	SaturationPlusFilter	1	2.0	饱和度增强
BW	WNBFilter	1	1.9	加权黑白转换
W	ImprovedWhiteBalanceFilter	3	1.7	白平衡校正

2.3 训练循环

核心流程（train.py 第 199-487 行）：

1. 从 ReplayMemory 采样 batch (image + state)
2. Agent forward: CNN 编码图像 → FC 输出 action logits → 采样滤波器 → 处理图像 → 输出 retouch + new_state
3. YOLOv3 forward: 分别计算原始图和处理后图的检测损失
4. $\text{Reward} = (\text{detect_input_loss} - \text{detect_retouch_loss}) \times 100 - \text{filter_usage_penalty}$
5. Value 网络评估 old_value 和 new_value, 计算 TD error
6. $\text{Agent Loss} = -Q_value + \text{surrogate}$ (熵正则化), $\text{Value Loss} = \text{MSE}(\text{advantage})$
7. Backward + optimizer step, 将 retouch 放回 ReplayMemory

关键技术细节：

ReplayMemory 容量 128, 存储 trajectory 片段, 每条 trajectory 最长 7 步

探索概率 exploration = 0.05, 训练时以 5% 概率随机选择 action

学习率余弦退火: $1.5e-5 \rightarrow 3.0e-8$ (10000 步)

filter_usage_penalty = 1.0: 重复使用同一滤波器扣 1.0 分, 鼓励多样化的滤波器组合

三、小改动实验

3.1 实验 A：数据集迁移 —— LOD → BIRDS 微调

3.1.1 动机

论文中 LOD-Agent 在低光目标检测数据集 (LOD) 上训练。本实验旨在验证：在特定场景（鸟类自然光摄影）上进行领域微调，能否进一步提升 ISP 策略对该场景的适配性，即 "domain matching 是否有益"。

3.1.2 改动内容

- (1) 创建 yolov3/data/bird.yaml，指向自建 BIRDS 数据集
- (2) 配置训练：batch_size=2 (8GB VRAM 限制), iterations=10000, val_freq=999999 (关闭验证以避免 OOM)
- (3) 使用预训练 LOD-Agent 权重初始化（而非从头训练），在 BIRDS 训练集上进行微调

训练过程：

显存占用稳定在 3.48GB，未出现 OOM

训练时长约 1 小时 41 分钟（10000 步）

Value loss 从 4.90 持续下降至 1.43，表明 value 网络收敛良好

Agent loss 从 -0.46 收敛至 -0.24，policy 趋于稳定

3.1.3 实验结果

模型	mAP50	mAP75	mAP50-95	Precision	Recall
LOD-Agent (预训练)	0.738	0.579	0.551	0.735	0.698
Bird-Agent (微调后)	0.738	0.566	0.529	0.762	0.683

滤波器使用分布对比：

滤波器	LOD-Agent	Bird-Agent	差异
G (Gamma)	0%	20.0%	Bird 独有
S+ (饱和度增强)	8.6%	20.4%	Bird $\uparrow$ 2.4 $\times$
Shr (锐化)	20.0%	15.4%	相近
Ct (对比度)	7.8%	14.6%	Bird $\uparrow$
CCM (色彩校正)	20.0%	6.0%	LOD $\downarrow$
T (色调曲线)	20.0%	10.2%	LOD $\downarrow$
BW (黑白)	20.0%	12.4%	LOD $\downarrow$
W (白平衡)	3.6%	0.2%	LOD $\downarrow$
NLM (降噪)	0%	0.8%	均极少使用
E (曝光)	0%	0%	均不使用

3.1.4 分析与讨论

核心发现：在 BIRDS 上微调并未提升检测精度，mAP50-95 反而从 0.551 降至 0.529。分析滤波器分布可以解释这一反直觉结果：

(1) 策略退化到捷径：Bird-Agent 将 40.4% 的选择集中在 G (Gamma) 和 S+ (SaturationPlus) 两个操作上，而 LOD-Agent 的策略更为均衡，5 个主要滤波器各占约 20%。Gamma + S+ 的组合通过增加对比度和饱和度来突出清晰目标，但对边缘/遮挡目标反而会压制其特征，导致 Recall 下降 (0.698 $\rightarrow$ 0.683)。

(2) 数据多样性 > Domain Match：LOD 数据集包含低光、多场景、多类别目标，迫使 Agent 学习鲁棒的 ISP 策略。BIRDS 数据集仅有 400 张白天鸟类照片，场景单一，Agent 容易找到 "加 Gamma 加饱和度即提高检测分" 的捷径，但这种策略缺乏泛化能力。

(3) Precision-Recall Trade-off：Bird-Agent 的 Precision 更高 (0.735 $\rightarrow$ 0.762) 但 Recall 更低 (0.698 $\rightarrow$ 0.683)，说明过强的对比度增强让 YOLOv3 对确认的目标更自信 (FP $\downarrow$)，但同时也让部分弱目标被淹没 (FN $\uparrow$)。

3.2 实验 B：减少 ISP 最大步数 (5 → 3)

3.2.1 动机

论文默认的 ISP 管线为 5 步迭代。减少步数可降低推理延迟，但可能损害检测精度。本实验旨在量化步数与精度之间的 trade-off。

3.2.2 改动内容

在评测时，仅修改 val_adaptiveisp.py 的 --steps 参数从 5 改为 3，其余设置（模型权重、数据集、batch_size）保持一致。无需重新训练。

3.2.3 实验结果

配置	mAP50	mAP75	mAP50-95	Precision	Recall	推理时间	加速
Bird-Agent steps=5	0.738	0.566	0.529	0.762	0.683	374.9ms	—
Bird-Agent steps=3	0.734	0.553	0.521	0.749	0.679	227.6ms	↑ 39.3%

3.2.4 分析

从 5 步减至 3 步，mAP50-95 仅下降 0.008（约 1.5%），但推理延迟减少 147.3ms（约 39%）。这表明：

- (1) 前 3 步的 ISP 处理贡献了大部分检测增益，第 4-5 步的边际收益很小。
- (2) 在对实时性有要求的应用场景中，steps=3 是更优的精度-速度平衡点。
- (3) 该结果也间接说明，Agent 训练的 5 步策略中，后两步可能主要执行了增益较低的"微调"操作。

四、失败案例与改进思路

4.1 失败案例分析

案例 1 — 策略过拟合 (Policy Overfitting):

Bird-Agent 在 BIRDS 上微调后，策略退化到严重依赖 Gamma + SaturationPlus 两个操作（占 40.4%）。这种捷径策略在训练集上可能有效（降低检测损失），但在同分布的验证集上反而逊于 LOD-Agent。这说明单一场景的数据集缺乏足够的"挑战"来迫使 RL agent 学习多元化的 ISP 策略。

案例 2 — Pseudo-label 质量问题:

实验中观察到部分图片（如图 4 所示）经 ISP 处理后，YOLOv3 成功检测到鸟类目标，但 YOLOv8 生成的 pseudo-label 中不存在该目标，导致正确检测被评测系统统计为 False Positive。这暴露了 pseudo-label 作为"参考答案"的固有局限：评测指标的上限受限于标签模型的性能，YOLOv8 漏标的目标在评测中永远是"错的"。



图 4.yolov8 与 agent+yolov3 的识别结果

案例 3 — Reward 振荡:

训练过程中 reward 持续在 -1.2 到 +1.2 之间大幅振荡。分析发现，filter_usage_penalty = 1.0 是主要扰动源——当 Agent 意外重复使用同一滤波器时，

直接扣除 1.0 分（相当于约 0.01 检测损失改善被抵消）。penalty 设置偏大使训练信号噪声较大，但另一方面也迫使 Agent 学习到多样化的滤波器组合。

4.2 改进思路

(1) 数据增强与多样性：在训练数据中引入多种退化（加噪、模糊、亮度抖动、色彩偏移），模拟 LOD 数据集的多样性，避免 Agent 学到场景相关的捷径策略。

(2) 标签质量提升：对 YOLOv8 的 pseudo-label 进行清理——用多模型 ensemble 投票、人工抽查高置信度差异样本、或使用更大的标注模型（如 YOLOv8x 替换 YOLOv8n）以提高召回率。

(3) Reward 设计调优：将 filter_usage_penalty 从 1.0 降至 0.3，或改为基于滤波器计算成本的加权惩罚（如对 NLM 罚更多，对 Gamma 罚更少），在策略多样性和训练稳定性之间取得更好平衡。

(4) 增加 ISP 操作：参考论文中提到的 Desaturation 模块，为低对比度场景增加柔和的饱和度降低操作，作为 SaturationPlus 的对偶操作，可能有助于改善 Recall。

(5) 更多对比实验：调整 reward 中 runtime penalty 的权重 ($\lambda = 0, 0.01, 0.05$)，分析 Agent 在检测质量与计算效率之间的权衡策略。

五、总结

本次实验完成了 AdaptiveISP 项目的环境搭建、代码理解、基线复现、数据集迁移微调和 ISP 步数消融实验。本次实验所使用的 500 张鸟类原始 ARW 图像均为自主拍摄，完成从原始 RAW 解码、弱监督伪标注到数据集划分的全流程构建，进一步加深了图像原始数据处理、弱监督目标检测数据集构建的实操认知。主要结论如下：

1. 在单场景小规模数据集上微调 RL-ISP agent 存在策略过拟合风险——Agent 倾向于找捷径操作（Gamma + SaturationPlus）而非学习鲁棒的多元化处理管线。训练数据的场景多样性比简单的 domain matching 对策略质量的影响更大。

2. 减少 ISP 步数可在几乎不影响检测精度的情况下显著提升推理速度（5→3 步仅损失 0.008 mAP50-95，加速 39%），为实际部署提供了精度-速度 trade-off 的参考。

3. Pseudo-label 的局限性是弱监督学习中的共性问题，在评测分析中需要保持批判意识——低 mAP 不一定代表 ISP 策略差，也可能源于标签本身的不完整。

4. 实验中遇到的问题（验证 OOM、数据集格式兼容）加深了对 PyTorch 显存管理和深度学习项目工程实践的理解。

附录 A 各组模型完整检测性能曲线

A.1 LOD-Agent (steps=5) 基线曲线

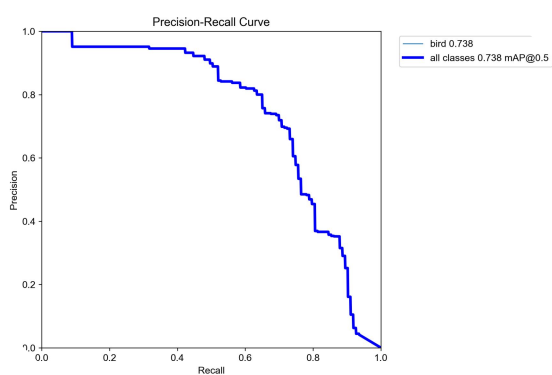


图 A1 PR 曲线

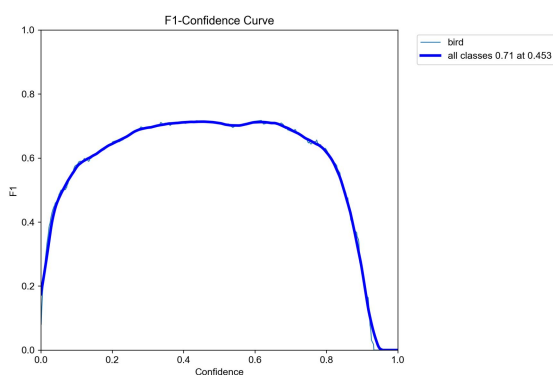


图 A2 F1-Conf 曲线

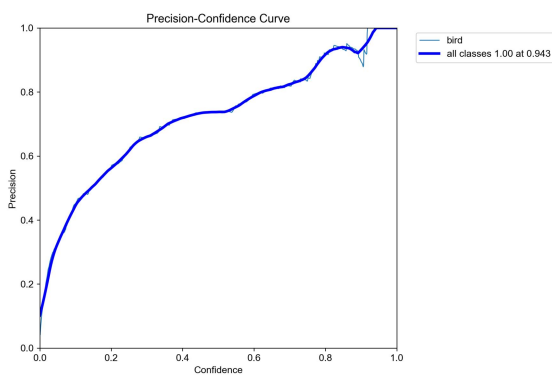


图 A3 Precision-Conf 曲线

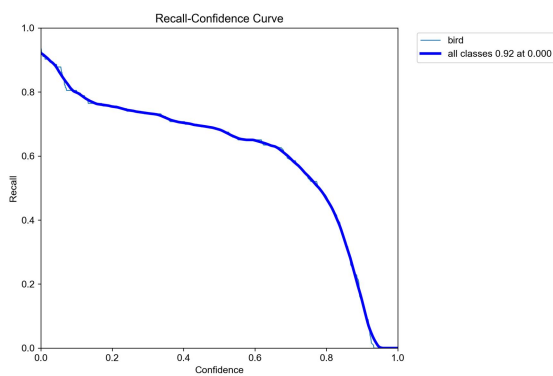


图 A4 Recall-Conf 曲线

A.2 Bird-Agent 微调后 steps=5

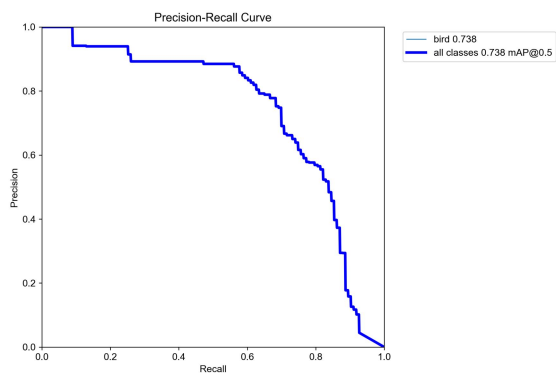


图 A5 PR 曲线

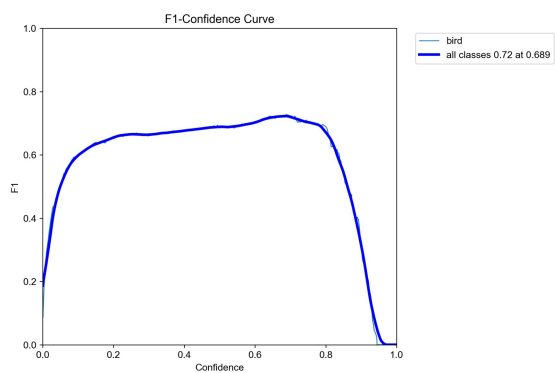


图 A6 F1-Conf 曲线

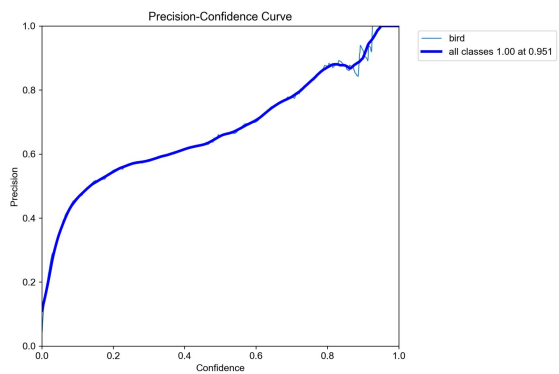


图 A7 Precision-Conf 曲线

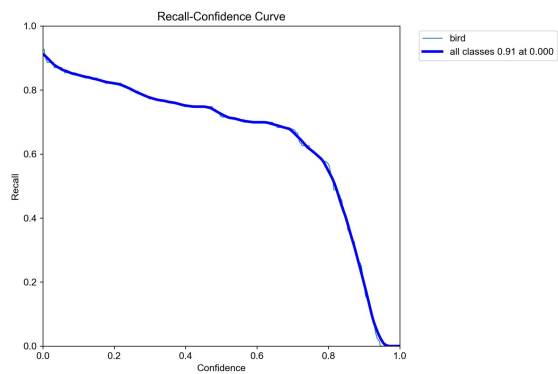


图 A8 Recall-Conf 曲线

A.3 Bird-Agent 微调后 steps=3

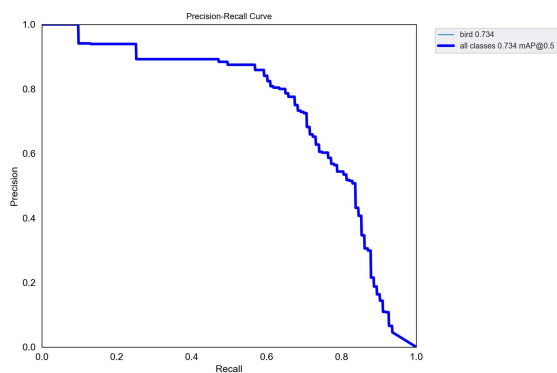


图 9 PR 曲线

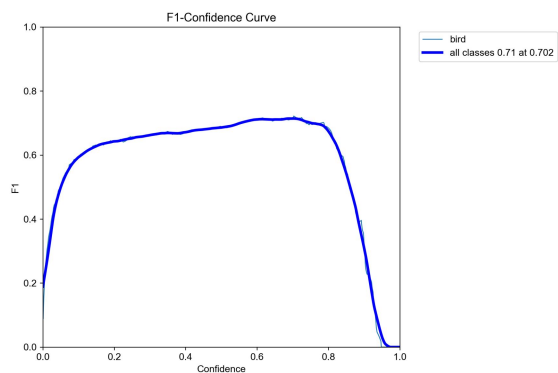


图 A10 F1-Conf 曲线

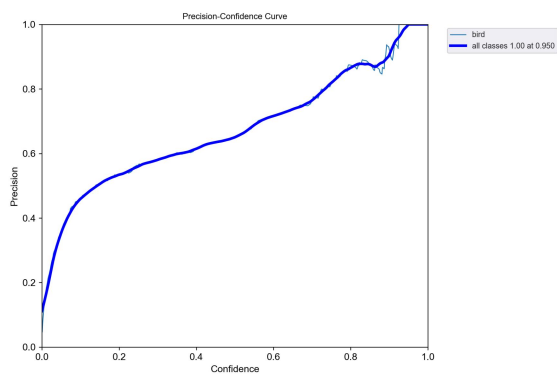


图 A11 Precision-Conf 曲线

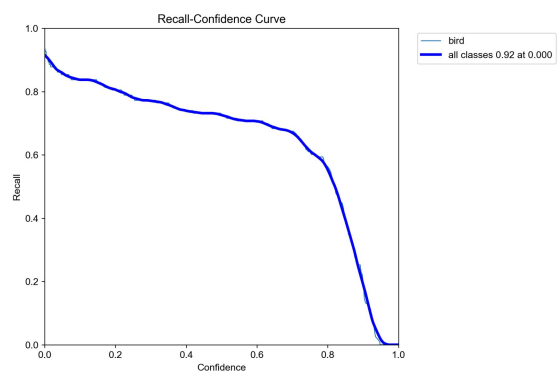


图 A12 Recall-Conf 曲线